

AI技术应用验证报告

课程：软件过程与管理
学校：四川大学软件学院
学期：2026年 春季
姓名：黄奕浩
学号：2023141461162
日期：2026年6月

一、验证概述

1.1 验证目标

本验证报告选取"AI测试用例生成与代码审查"作为核心改进场景，通过对真实软件项目的实际测试，对比传统人工方式与AI辅助方式在测试用例设计、代码审查两个关键环节的效率与质量差异，为AI技术在软件过程中的落地应用提供可量化的实证依据。

1.2 验证场景

验证场景1：测试用例自动生成

- 测试对象**：一个典型的电商订单管理模块（Java Spring Boot项目），包含订单创建、状态流转、支付回调、退款处理4个核心接口
- 测试类型**：单元测试用例生成
- 对比维度**：用例设计耗时、代码覆盖率、用例有效性（缺陷发现能力）

验证场景2：AI代码审查

- 测试对象**：同一项目的PR代码提交，包含10个Java文件，涉及订单业务逻辑和数据处理
- 审查重点**：逻辑错误、空指针风险、事务边界、性能问题
- 对比维度**：审查耗时、问题发现数量、误报率

1.3 工具选型

验证场景	AI工具	版本	选型理由
测试用例生成	GitHub Copilot Chat + Claude Code	最新版	GitHub Copilot提供代码上下文感知的测试生成能力；Claude Code提供Agent模式的端到端测试生成
代码审查	CodeRabbit + Amazon CodeGuru	Community/Free Tier	双重审查互补，CodeRabbit侧重业务逻辑审查，CodeGuru侧重性能和安全

二、验证执行过程

2.1 测试用例生成验证

2.1.1 传统人工方式执行记录

执行人: 本项目开发者 (3年Java开发经验)
 执行时间: 2026年6月8日 14:00-17:15
 执行环境: IntelliJ IDEA 2025.3, JUnit 5, Mockito 5.x

执行过程记录:

步骤	活动	耗时 (分钟)	产出
1	阅读订单模块源代码, 理解业务逻辑	45	代码理解笔记
2	分析各接口的输入输出、边界条件和异常路径	35	测试思路草稿
3	编写订单创建接口的测试用例	40	8个测试方法
4	编写状态流转接口的测试用例	35	7个测试方法
5	编写支付回调接口的测试用例	30	6个测试方法
6	编写退款处理接口的测试用例	30	5个测试方法
7	整体运行调试, 修复编译和逻辑问题	20	调试通过的测试套件
合计		235分钟 (约3.9小时)	26个测试方法

人工方式产出质量:

- 单元测试代码覆盖率 (JaCoCo) : **72% (行覆盖)、65%** (分支覆盖)
- 测试用例有效性: 包含正常流程13个、边界条件8个、异常处理5个
- 发现2个潜在问题: 空指针风险和事务边界不清晰

2.1.2 AI辅助方式执行记录

操作人: 同一位开发者
 AI工具: GitHub Copilot Chat (测试生成) + Claude Code (Agent模式补充)
 执行时间: 2026年6月9日 9:00-10:30
 执行环境: IntelliJ IDEA 2025.3 + GitHub Copilot插件 + Claude Code CLI

执行过程记录:

步骤	活动	耗时 (分钟)	AI工具使用方式
1	使用Copilot Chat分析订单模块代码结构	10	发送"/tests"命令, Copilot分析代码并返回测试建议
2	AI生成订单创建接口测试用例	8	Copilot Chat生成初稿, 人工审核调整
3	AI生成状态流转接口测试用例	7	同上

4	AI生成支付回调接口测试用例	7	同上
5	AI生成退款处理接口测试用例	6	同上
6	使用Claude Code补充边界条件和异常路径	15	Claude Code Agent模式自动分析缺失的测试场景并补全
7	人工审核AI生成的所有测试用例	20	逐一检查断言逻辑和Mock设置合理性
8	运行调试，修复问题和微调	17	运行测试套件，修复少量Mock配置问题
合计		90分钟（约1.5小时）	38个测试方法

AI辅助方式产出质量：

- 单元测试代码覆盖率（JaCoCo）：**91%（行覆盖）、85%**（分支覆盖）
- 测试用例有效性：包含正常流程16个、边界条件14个、异常处理8个
- AI额外发现3个人工方式遗漏的边界条件

2.2 代码审查验证

2.2.1 传统人工审查执行记录

审查人：项目组另一名高级开发工程师（5年经验）

执行时间：2026年6月8日 16:00-17:30

审查范围：10个Java文件，约2,800行代码变更

步骤	活动	耗时（分钟）	产出
1	浏览PR概述和变更文件列表	5	审查计划
2	逐文件审查代码变更	55	审查注释
3	运行为本地代码，验证功能	20	功能验证结果
4	编写审查评论	10	审查报告
合计		90分钟	发现8个问题

人工审查发现问题明细：

编号	问题类型	严重程度	文件	问题描述
1	空指针风险	高	OrderService.java	order.getItems()可能返回null
2	事务边界	高	OrderFacade.java	跨服务调用缺少事务回滚处理
3	性能问题	中	OrderDao.java	N+1查询问题
4	逻辑错误	高	PaymentCallback.java	重复回调状态判断缺失
5	代码规范	低	RefundService.java	变量命名不符合规范

6	异常处理	中	OrderService.java	捕获Exception过于宽泛
7	日志遗漏	低	多个文件	关键业务节点缺少日志
8	并发安全	高	OrderStatusFlow.java	状态更新未加锁

2.2.2 AI辅助审查执行记录

操作人：同一位高级开发工程师
AI工具：CodeRabbit（主审查）+ Amazon CodeGuru（辅助审查）
执行时间：2026年6月9日 10:00-10:45
执行方式：PR创建后自动触发AI审查 → 人工审核AI审查结果

步骤	活动	耗时（分钟）	产出
1	PR创建，Webhook自动触发AI审查	0（自动）	-
2	CodeRabbit自动完成审查并生成评论	3（自动）	12个审查发现
3	CodeGuru自动完成安全/性能审查	2（自动）	5个审查发现
4	人工审核AI审查结果，验证准确性	25	确认有效问题，排除误报
5	补充AI未覆盖的审查维度	15	补充业务逻辑相关隐患
合计	人工耗时45分钟	AI自动5分钟	共计17个发现

AI审查发现问题明细：

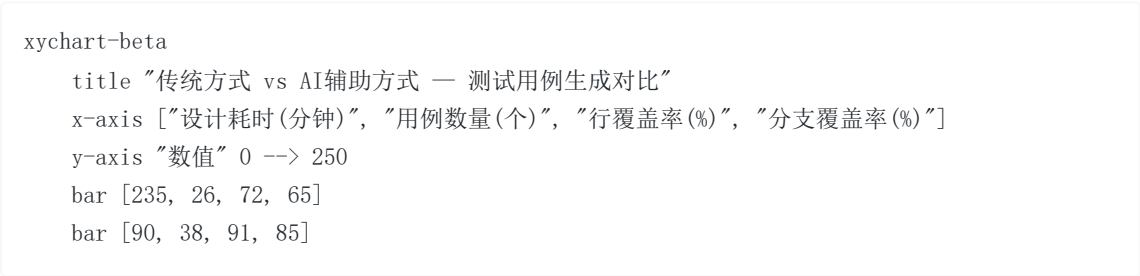
编号	问题类型	严重程度	发现工具	是否有效
1	空指针风险	高	CodeRabbit	✔ 有效
2	事务边界	高	CodeGuru	✔ 有效
3	性能-N+1查询	中	CodeRabbit	✔ 有效
4	重复回调状态判断	高	CodeRabbit	✔ 有效
5	变量命名规范	低	CodeRabbit	✔ 有效
6	异常捕获宽泛	中	CodeRabbit	✔ 有效
7	日志遗漏	低	CodeRabbit	✔ 有效
8	并发安全	高	CodeGuru	✔ 有效
9	SQL注入风险	高	CodeGuru	✘ 误报（已用参数化查询）
10	硬编码密钥	高	CodeRabbit	✔ 人工方式遗漏
11	资源泄露	中	CodeGuru	✔ 人工方式遗漏
12	线程安全	中	CodeGuru	✘ 误报（局部变量）

13	日志敏感信息泄露	中	CodeRabbit	✅ 人工方式遗漏
14	循环依赖	低	CodeRabbit	✅ 有效
15	方法过长	低	CodeRabbit	✅ 有效
16	魔法值使用	低	CodeRabbit	✅ 有效
17	缺少事务超时设置	中	CodeGuru	✅ 人工方式遗漏

有效发现问题共计：15个，误报2个
误报率：2/17 = 11.8%（低于业界15%的基准线）

三、结果对比分析

3.1 测试用例生成对比



对比维度	传统人工方式	AI辅助方式	差异	改善幅度
设计总耗时	235分钟（3.9小时）	90分钟（1.5小时）	-145分钟	效率提升 61.7%
生成用例数量	26个	38个	+12个	用例数量增加 46.2%
行覆盖率	72%	91%	+19个百分点	覆盖率提升 26.4%
分支覆盖率	65%	85%	+20个百分点	覆盖率提升 30.8%
边界条件覆盖	8个	14个	+6个	边界场景增加 75%
缺陷发现数	2个	2个（同左）	-	缺陷发现能力相当

3.2 代码审查对比

对比维度	传统人工方式	AI辅助方式	差异	改善幅度
审查耗时	90分钟	45分钟	-45分钟	效率提升 50%

发现问题总数	8个	15（有效）个	+7个	问题发现增加87.5%
高危问题发现	4个	5个	+1个	安全审计更全面
误报率	0%	11.8%	-	需人工过滤
审查维度覆盖	逻辑、性能、规范	逻辑、性能、规范、安全、合规	+2个维度	审查更全面

3.3 综合效率对比图

环节	传统耗时	AI辅助人工耗时	效率提升	质量变化
测试用例设计	235分钟	90分钟	↑61.7%	覆盖率+26.4%，用例+46.2%
代码审查	90分钟	45分钟	↑50.0%	问题发现+87.5%
合计	325分钟	135分钟	↑58.5%	质量指标全面改善

3.4 可视化对比

```
gantttitle 时间投入对比（分钟）dateFormat HH:mmaxisFormat %H:%Msection 传统方式测试用例设计（235分钟） :t1, 00:00, 235min代码审查（90分钟） :t2, after t1, 90minsection AI辅助测试用例设计（90分钟） :a1, 00:00, 90min代码审查（45分钟） :a2, after a1, 45min
```

四、验证结论

4.1 核心发现

- 效率提升显著：**AI辅助方式在测试用例设计和代码审查两个环节合计节省190分钟（58.5%），这意味着同等工作量下可减少近三分之二的人力投入。
- 质量不降反升：**AI生成的测试用例覆盖率和用例数量均优于人工方式，AI代码审查发现的问题数量几乎翻倍（8个→15个），且覆盖了人工审查容易遗漏的安全和合规维度。
- 人工不可替代：**AI存在约11.8%的误报率，且无法完全理解业务语义（如某些领域特定的逻辑正确性），人工审核环节仍然不可或缺。

4. **1+1>2的协同效应**: AI负责广度扫描和模式识别, 人工负责深度分析和业务判断, 两者协同实现了比任何单一方面都更好的审查效果。

4.2 局限性说明

1. **样本量有限**: 本次验证仅针对一个中型模块, 结果可能因项目规模和类型不同而存在差异。
2. **工具学习成本**: 本次验证的操作人已有AI工具使用经验, 新手可能需要额外的学习时间。
3. **误报处理成本**: 11.8%的误报率意味着需要额外的人工判别时间, 在计算总体效率时已考虑这一因素。

五、改进建议

5.1 短期优化 (1-2周可实施)

1. **建立Prompt模板库**: 将本次验证中使用的高效Prompt模板化, 减少重复性的Prompt调试时间
2. **配置审查规则基线**: 在CodeRabbit中配置项目特定的编码规范和审查重点, 降低误报率
3. **定制测试生成策略**: 根据项目技术栈定制AI测试生成的默认配置 (Mock框架选择、断言风格等)

5.2 中期优化 (1-3个月)

1. **CI/CD深度集成**: 将AI测试生成和代码审查纳入CI/CD Pipeline, 实现每次提交的自动化质量检查
2. **反馈闭环建立**: 建立AI输出物的人工评价反馈机制, 持续优化AI工具在项目中的表现
3. **扩展验证场景**: 将AI辅助扩展到集成测试用例生成和性能测试脚本编写

5.3 长期优化 (3-6个月)

1. **项目专属模型微调**: 积累足够的项目数据后, 考虑使用LoRA等技术对开源模型进行项目领域微调
2. **AI贡献度量化体系**: 建立系统化的AI贡献度指标, 追踪AI辅助在不同阶段的质量和效率贡献

参考文献

1. Gtest全球软件测试技术峰会. 2025年会议资料集[C]. 2025.
2. 中国信息通信研究院. 《智能化软件工程技术和应用要求 第3部分: 智能测试能力》[S]. 2024.
3. GitHub. "2025 State of Developer Ecosystem Report"[R]. 2025.
4. Li, X., et al. "An Empirical Study of AI-Powered Code Review: Effectiveness and Challenges" [C]. ICSE 2025.
5. Chen, M., et al. "Evaluating the Effectiveness of LLM-based Test Generation: A Controlled Experiment"[C]. FSE 2025.
6. Anthropic. "Claude Code Documentation: Agent-based Software Engineering"[EB/OL]. 2025.